

Relazione PDBP 2023

Luxmetro con fotodiodo per luce continua

Numero del gruppo: 22

- Traverso, Matteo, Giovanni, 322816
- Maffei, Simone, 324097
- Tarditi, Valerio, 322406

Specifiche del progetto e funzionamento teorico

Il dispositivo realizzato nell'esperienza di laboratorio dovrà, da specifiche, misurare circa ogni 1000 ms la tensione di batteria, simulata mediante un partitore realizzato con un trimmer, e con la stessa frequenza, l'intensità luminosa espressa in klux da rappresentare su tre display a sette segmenti. Sulla basetta sarà presente anche un led, il quale, a seconda del superamento o meno di una soglia prestabilita di tensione pari a 4,2 V, ci si aspetta segnali all'utente un'alimentazione insufficiente mediante la sua accensione.

Nello specifico il sensore utilizzato per la realizzazione del luxmetro è il fotodiodo SFH203, inserito in un opportuno circuito di condizionamento che gli consente di tradurre la foto-corrente prodotta al variare dell'intensità luminosa in tensione in uscita all'amplificatore.

Si è deciso per alimentare la breadboard su cui è montato il progetto di utilizzare l'uscita a 5 V della scheda ARDUINO UNO, la quale a sua volta è stata precedentemente alimentata con una tensione di 7 V, fornita dal primo canale dell'alimentatore stabilizzato. Si è scelta come tensione di alimentazione di ARDUINO UNO il valore di 7 V nel pin di Vin in quanto indicata nel datasheet del componente come minima tensione ammessa su quella piedinatura al fine di garantire il corretto funzionamento della scheda. Inoltre, come sarà spiegato in seguito, la tensione scelta come riferimento dell'ADC per eseguire i campionamenti necessari viene forzata nel pin di Aref dal secondo canale dell'alimentatore stabilizzato, impostato ad una tensione di 3,3 V.

Per quanto riguarda l'amplificatore di transresistenza utilizzato per condizionare il fotodiodo, è stato fornito l'amplificatore operazionale TL081, che a causa dell'elevato offset altamente variabile, ha pregiudicato la portata del luxmetro realizzato, che si prevede non vada oltre i 6,55 klux, di fronte ad una scelta di progetto iniziale che comprendeva una portata di 9,99 klux e l'impiego di un amplificatore OPA344, il quale possiede un offset decisamente minore, circa 100 mV rispetto ad 1,50 V del TL081, e che avrebbe permesso di sfruttare la quasi totalità della dinamica garantendo in questo modo anche una sensibilità maggiore.

L'utilizzo di tale amplificatore ha poi comportato la scelta di una R2 pari a 12 k Ω contrariamente ai 560 Ω previsti, in modo da non portare immediatamente in saturazione il componente. Come resistore di feedback, al fine di ottenere una sensibilità opportuna, si è scelto un valore di 4100 Ω , realizzato mediante una serie tra le resistenze di 3900 Ω e 270 Ω .

In laboratorio, tramite il multimetro, si sono misurati i valori reali delle resistenze in serie, rispettivamente 3820 Ω e 264 Ω , ottenendo così un resistore di feedback con una resistenza complessiva di 4084 Ω .

È stata eseguita una taratura tramite metodo end-points con l'impiego di un luxmetro portatile, ricavando la sensibilità reale secondo le formule riportate in seguito. Si è scelto questo metodo di taratura perché l'unico in grado di garantire un risultato accettabile a fronte di variazioni di misura sperimentale anche molto evidenti dovute agli strumenti utilizzati ed agli errori derivanti dal movimento dell'operatore.

Calcoli e rilievi sperimentali

■ Calcoli R feedback:

$$R_F = \frac{V_{AL} - V_{OFFSET}}{S \cdot klux} = \frac{5V - 1.52V}{80 \mu A / klux \cdot 9.99 klux} = 4355 \Omega$$

Si è scelto un valore inferiore di resistenza pari a 4100Ω , in seguito alla calibrazione.

■ Calcoli clock e timer counter:

Inizialmente si è deciso di dividere la frequenza del clock interno per 16, in modo tale da ridurre i consumi non necessari, per avere una misurazione ogni secondo è stato deciso di avere un interrupt ogni 20,48 ms, di seguito vengono elencati i calcoli effettuati:

$$f_{clock} = \frac{16 MHz}{16} = 1 MHz$$

$$f_{TC} = \frac{1 MHz}{1024} = 976,6 Hz$$

$$T_{TC} = 1024 \mu s$$

$$T_{int} = T_{TC} \cdot 20 \cong 20 ms$$

$$var_D = \frac{1000 ms}{20 ms} = 50$$

■ Calcoli taratura:

Per la taratura si è simulata una condizione di fondo scala: con una torcia è stato illuminato sia il circuito montato su breadboard che il luxmetro portatile fino ad avere su quest'ultimo un'intensità luminosa di circa $(9,80 \div 9,99)$ klux, andando a rilevare con il multimetro la tensione presente all'uscita del circuito di condizionamento di circa 2,80 V, valore nel quale è compresa la tensione di offset preesistente, eliminata nella look-up table tramite l'azzeramento dei primi 115 livelli.

Come già precedentemente spiegato è stata utilizzata una $R_F^{Reale} = 4084 \Omega$.

Si è calcolata una sensibilità reale pari a:

$$S = \frac{2,80 V}{4084 \Omega \cdot 9,99 klux} \cong 68 \mu A / klux$$

Con la quale sono stati poi tarati tutti i valori nella look-up table.

La taratura, in ogni caso, non risulta essere molto accurata poiché la torcia non ha illuminato sia il fotodiodo che il luxmetro portatile dalla stessa distanza e con la medesima angolazione per la difficoltà da parte dell'operatore di creare e mantenere un ambiente di misura stabile nel tempo e con caratteristiche omogenee; ciò ha portato ad errori che potrebbero essere facilmente corretti con strumenti più stabili, modificando poi la look-up table.

■ Calcoli lut:

$$V_q = \frac{V_{Ref}}{2^N} = \frac{3,3 V}{2^8} = 12,9 mV$$

$$V = V_q \cdot n \quad \text{con } n \text{ numero dell'indice della look up table}$$

Poiché è presente un offset variabile di circa 1,49 V, nella look-up table verranno messi forzatamente a 0 perché inutilizzabili i primi 116 livelli.

$$n_0 = \frac{1,49 V}{12,9 mV} = 115,5 \cong 116 \text{ livelli}$$

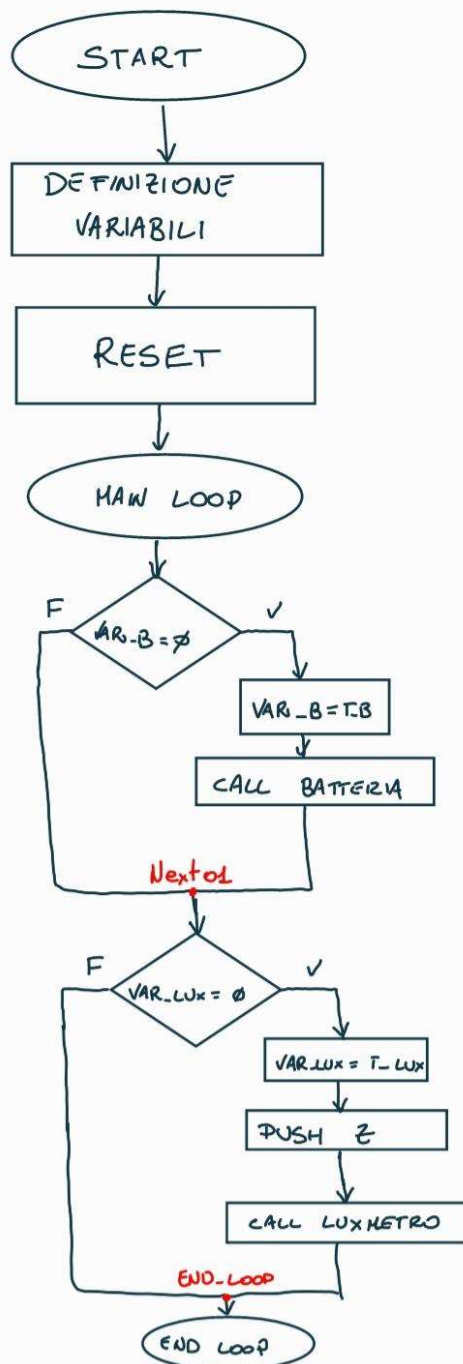
$n = 1$ in realtà corrisponderà quindi al 117esimo livello e quindi arriverà fino a $n_{max} = 256 - 116 = 140$ con un passo per ogni livello pari a 12,9 mV.

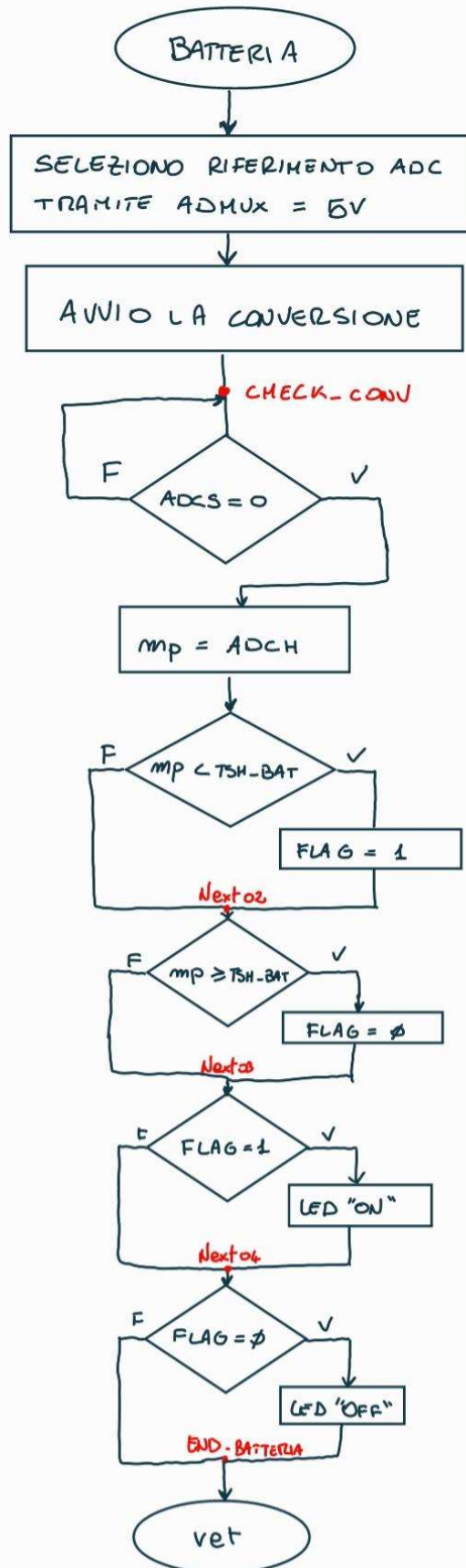
Per questo motivo è stata raggiunta una portata massima di 6,55 klux a fronte dei 9,99 klux inizialmente selezionati.

La conversione da mV a klux è stata eseguita sulla base della seguente formula:

$$klux = \frac{V}{R_F \cdot S} \quad \text{considerando la sensibilità reale precedentemente ricavata dalla taratura } S = 68 \mu A/klux.$$

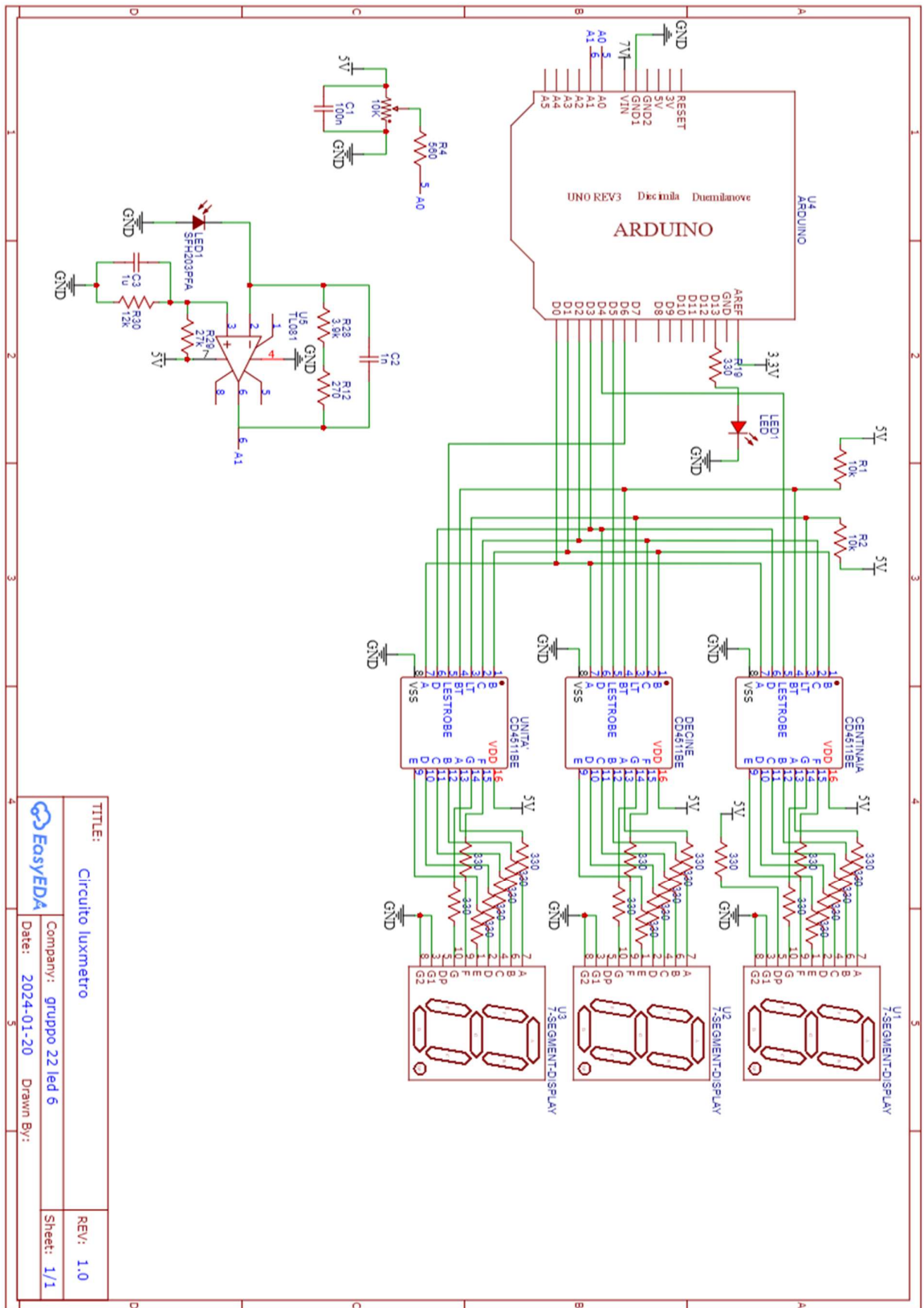
Flowchart del codice



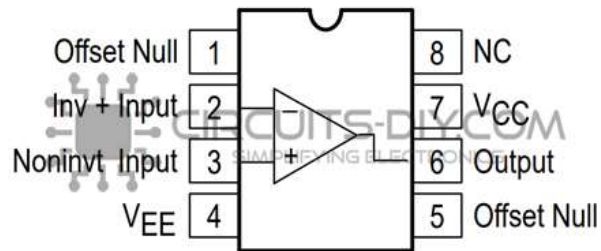




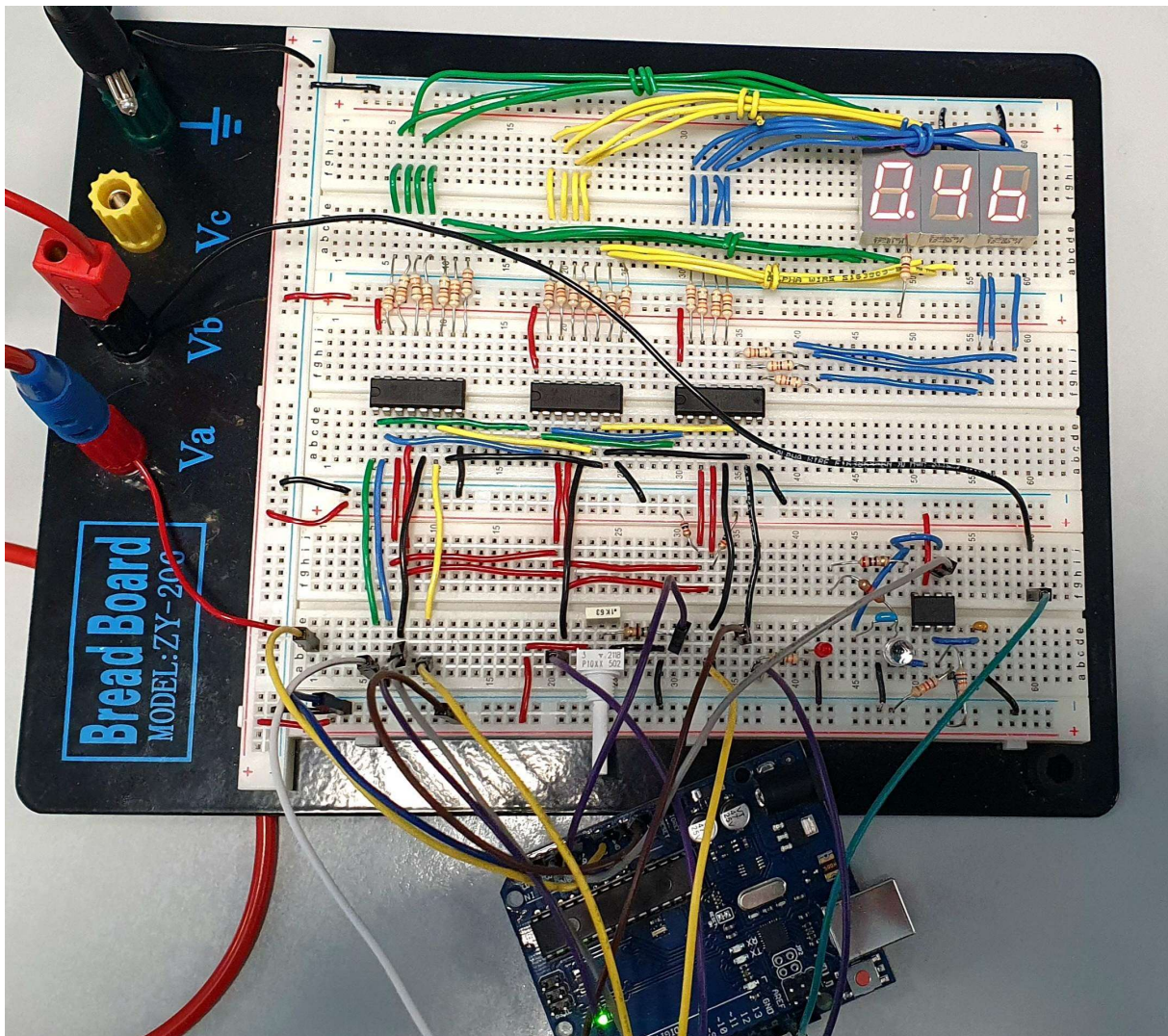
Schematico del circuito

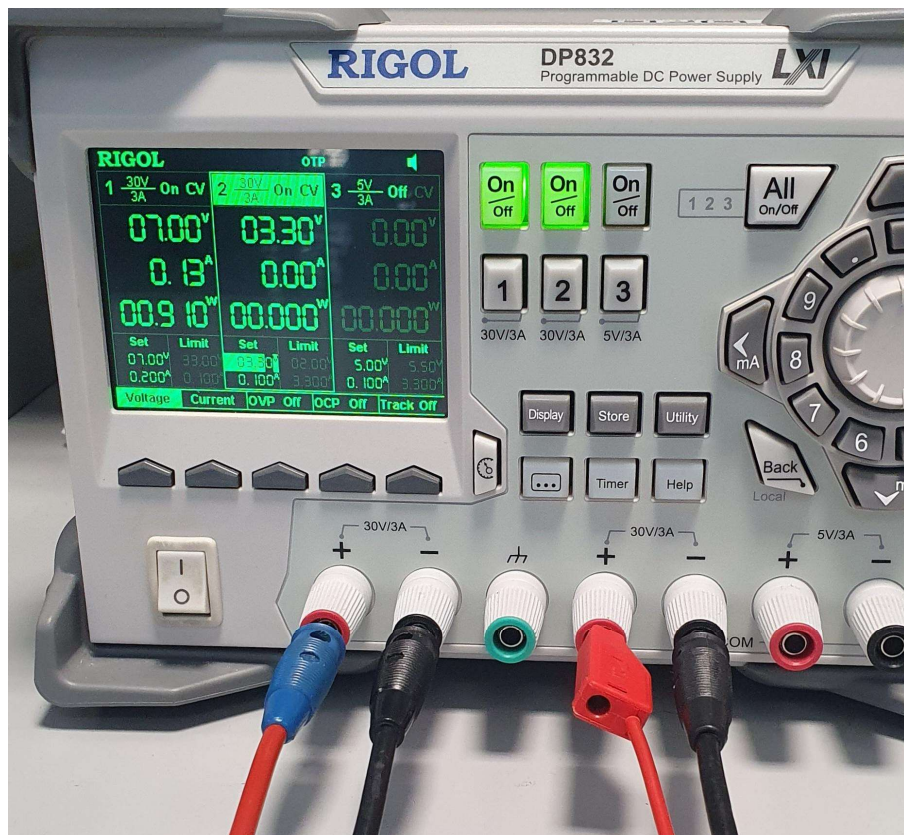
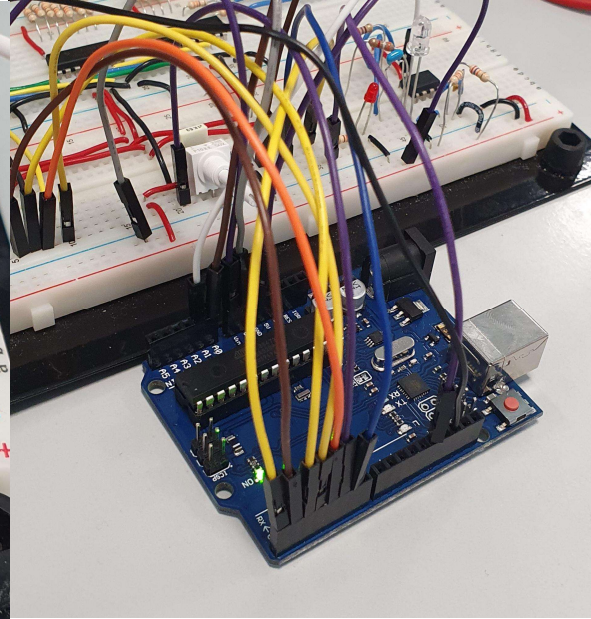
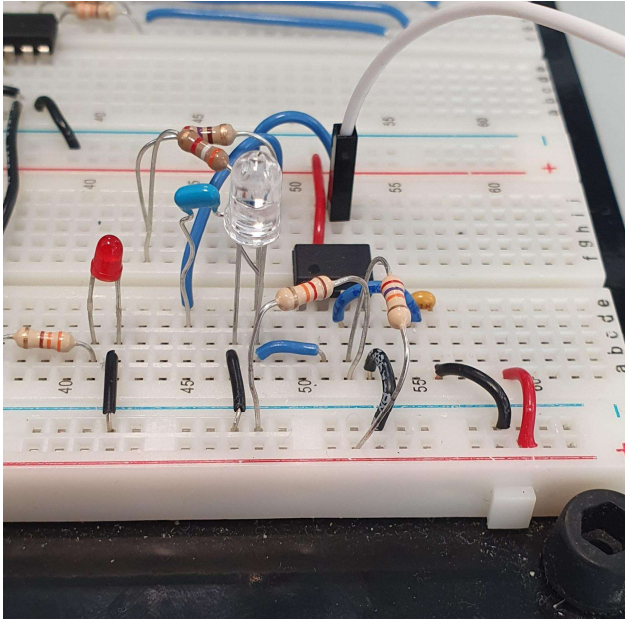


TL081 Pinout



Immagini del circuito montato su basetta e dei canali utilizzati dell'alimentatore stabilizzato






```
;
; Firmware_Finale.asm
;
; Created: 24/01/2024 19:01:51
; Author : matte
;
;
; Luxmetro_Codice01.asm
;
; Created: 20/01/2024 11:56:34
; Author : Gruppo 22 LED 6
;
;
; Definizione dei registri utilizzati
.DEF mp = R16 ; registro di lavoro (generico)
.DEF mp1 = R17 ; registro di lavoro secondario (generico)
.DEF var_B = R18 ; var_check_B è la variabile di batteria ↗
    decrementata dalla subroutine di risposta ad interrupt del timercounter
.DEF var_LUX = R19 ; var_ckeck_LUX è la variabile del luxmetro ↗
    decrementata dalla subroutine di risposta ad interrupt del timercounter
    ; quando arrivano a 0 viene attivato un ↗
        campionamento dell'ADC
.DEF flags = R20 ; registro che consente di disporre di 8 flag ↗
    binari (in questo programma usato solo il bit 0)
;
.EQU T_B = 50 ; costante che definisce l'intervallo di ↗
    campionamento della tensione di batteria in multipli di 10ms
.EQU T_LUX = 50 ; costante che definisce l'intervallo di ↗
    campionamento della tensione generata dal fotodiodo in multipli di 10 ms
.EQU TSH_BAT = 217 ; costante che definisce la tensione di soglia di ↗
    4,2 V considerando un riferimento a 5 V e 256
    ; valori possibili (da 0 a 255) della conversione ↗
        su 8 bit
;
; Crea tabella nella flash. In questo caso la tabella ha 256 righe consistenti ↗
    ognuna in 4 byte
; valori possibili (da 0 a 255) della conversione su 8 bit. La prima cella ↗
    rappresenta l'uscita dell'ADC e non è utilizzata
; la seconda la cifra delle centinaia da mandare al display; la terza la cifra ↗
    delle decine; la quarta la cifra delle unità.
; In uscita avremo quindi l'intensità luminosa su 3 display fino alla portata ↗
    di 9.99 klux, con risoluzione di 0.01 klux.
.CSEG
.ORG 0x1FFF ; Definisce l'indirizzo d'inizio della memoria ↗
    flash nella quale verrà scritta la tabella
;
tabella:
.db 0,0,0,0
.db 1,0,0,0
```

```
.db 2,0,0,0
.db 3,0,0,0
.db 4,0,0,0
.db 5,0,0,0
.db 6,0,0,0
.db 7,0,0,0
.db 8,0,0,0
.db 9,0,0,0
.db 10,0,0,0
.db 11,0,0,0
.db 12,0,0,0
.db 13,0,0,0
.db 14,0,0,0
.db 15,0,0,0
.db 16,0,0,0
.db 17,0,0,0
.db 18,0,0,0
.db 19,0,0,0
.db 20,0,0,0
.db 21,0,0,0
.db 22,0,0,0
.db 23,0,0,0
.db 24,0,0,0
.db 25,0,0,0
.db 26,0,0,0
.db 27,0,0,0
.db 28,0,0,0
.db 29,0,0,0
.db 30,0,0,0
.db 31,0,0,0
.db 32,0,0,0
.db 33,0,0,0
.db 34,0,0,0
.db 35,0,0,0
.db 36,0,0,0
.db 37,0,0,0
.db 38,0,0,0
.db 39,0,0,0
.db 40,0,0,0
.db 41,0,0,0
.db 42,0,0,0
.db 43,0,0,0
.db 44,0,0,0
.db 45,0,0,0
.db 46,0,0,0
.db 47,0,0,0
.db 48,0,0,0
.db 49,0,0,0
.db 50,0,0,0
```

```
.db 51,0,0,0
.db 52,0,0,0
.db 53,0,0,0
.db 54,0,0,0
.db 55,0,0,0
.db 56,0,0,0
.db 57,0,0,0
.db 58,0,0,0
.db 59,0,0,0
.db 60,0,0,0
.db 61,0,0,0
.db 62,0,0,0
.db 63,0,0,0
.db 64,0,0,0
.db 65,0,0,0
.db 66,0,0,0
.db 67,0,0,0
.db 68,0,0,0
.db 69,0,0,0
.db 70,0,0,0
.db 71,0,0,0
.db 72,0,0,0
.db 73,0,0,0
.db 74,0,0,0
.db 75,0,0,0
.db 76,0,0,0
.db 77,0,0,0
.db 78,0,0,0
.db 79,0,0,0
.db 80,0,0,0
.db 81,0,0,0
.db 82,0,0,0
.db 83,0,0,0
.db 84,0,0,0
.db 85,0,0,0
.db 86,0,0,0
.db 87,0,0,0
.db 88,0,0,0
.db 89,0,0,0
.db 90,0,0,0
.db 91,0,0,0
.db 92,0,0,0
.db 93,0,0,0
.db 94,0,0,0
.db 95,0,0,0
.db 96,0,0,0
.db 97,0,0,0
.db 98,0,0,0
.db 99,0,0,0
```

```
.db 100,0,0,0
.db 101,0,0,0
.db 102,0,0,0
.db 103,0,0,0
.db 104,0,0,0
.db 105,0,0,0
.db 106,0,0,0
.db 107,0,0,0
.db 108,0,0,0
.db 109,0,0,0
.db 110,0,0,0
.db 111,0,0,0
.db 112,0,0,0
.db 113,0,0,0
.db 114,0,0,0
.db 115,0,0,0
.db 116,0,0,5
.db 117,0,0,9
.db 118,0,1,4
.db 119,0,1,9
.db 120,0,2,3
.db 121,0,2,8
.db 122,0,3,3
.db 123,0,3,7
.db 124,0,4,2
.db 125,0,4,6
.db 126,0,5,1
.db 127,0,5,6
.db 128,0,6,0
.db 129,0,6,5
.db 130,0,7,0
.db 131,0,7,4
.db 132,0,7,9
.db 133,0,8,4
.db 134,0,8,8
.db 135,0,9,3
.db 136,0,9,8
.db 137,1,0,2
.db 138,1,0,7
.db 139,1,1,1
.db 140,1,1,6
.db 141,1,2,1
.db 142,1,2,5
.db 143,1,3,0
.db 144,1,3,5
.db 145,1,3,9
.db 146,1,4,4
.db 147,1,4,9
.db 148,1,5,3
```

.db 149,1,5,8
.db 150,1,6,3
.db 151,1,6,7
.db 152,1,7,2
.db 153,1,7,7
.db 154,1,8,1
.db 155,1,8,6
.db 156,1,9,0
.db 157,1,9,5
.db 158,2,0,0
.db 159,2,0,4
.db 160,2,0,9
.db 161,2,1,4
.db 162,2,1,8
.db 163,2,2,3
.db 164,2,2,8
.db 165,2,3,2
.db 166,2,3,7
.db 167,2,4,2
.db 168,2,4,6
.db 169,2,5,1
.db 170,2,5,5
.db 171,2,6,0
.db 172,2,6,5
.db 173,2,6,9
.db 174,2,7,4
.db 175,2,7,9
.db 176,2,8,3
.db 177,2,8,8
.db 178,2,9,3
.db 179,2,9,7
.db 180,3,0,2
.db 181,3,0,7
.db 182,3,1,1
.db 183,3,1,6
.db 184,3,2,1
.db 185,3,2,5
.db 186,3,3,0
.db 187,3,3,4
.db 188,3,3,9
.db 189,3,4,4
.db 190,3,4,8
.db 191,3,5,3
.db 192,3,5,8
.db 193,3,6,2
.db 194,3,6,7
.db 195,3,7,2
.db 196,3,7,6
.db 197,3,8,1

```
.db 198,3,8,6
.db 199,3,9,0
.db 200,3,9,5
.db 201,3,9,9
.db 202,4,0,4
.db 203,4,0,9
.db 204,4,1,3
.db 205,4,1,8
.db 206,4,2,3
.db 207,4,2,7
.db 208,4,3,2
.db 209,4,3,7
.db 210,4,4,1
.db 211,4,4,6
.db 212,4,5,1
.db 213,4,5,5
.db 214,4,6,0
.db 215,4,6,5
.db 216,4,7,0
.db 217,4,7,4
.db 218,4,8,3
.db 219,4,8,8
.db 220,4,9,2
.db 221,4,9,7
.db 222,5,0,2
.db 223,5,0,6
.db 224,5,1,1
.db 225,5,1,6
.db 226,5,2,0
.db 227,5,2,5
.db 228,5,3,0
.db 229,5,3,4
.db 230,5,3,9
.db 231,5,4,3
.db 232,5,4,8
.db 233,5,5,3
.db 234,5,5,7
.db 235,5,6,2
.db 236,5,6,7
.db 237,5,7,1
.db 238,5,7,6
.db 239,5,8,0
.db 240,5,8,5
.db 241,5,9,0
.db 242,5,9,5
.db 243,5,9,9
.db 244,6,0,4
.db 245,6,0,9
.db 246,6,1,3
```

```
.db 247,6,1,8
.db 248,6,2,2
.db 249,6,2,7
.db 250,6,3,2
.db 251,6,3,6
.db 252,6,4,1
.db 253,6,4,6
.db 254,6,5,1
.db 255,6,5,5
;
.CSEG
.ORG 0x0000 ; Defisce la posizione d'inizio del codice del programma all'indirizzo 0x0000
;
; INTERRUPT VECTORS FOLLOW
;
    jmp RESET ; vector 1: Reset Handler
    jmp EXT_INT0 ; vector 2: IRQ0 Handler
    jmp EXT_INT1 ; vector 3: IRQ1 Handler
    jmp PCINTR0 ; vector 4: PCINT0 Handler
    jmp PCINTR1 ; vector 5: PCINT1 Handler
    jmp PCINTR2 ; vector 6: PCINT2 Handler
    jmp WDT ; vector 7: Watchdog timer handler
    jmp TIM2_COMPA ; vector 8: Timer2 Compare A handler
    jmp TIM2_COMPB ; vector 9: Timer2 compare B handler
    jmp TIM2_OVF ; vector 10: Timer2 Overflow Handler
    jmp TIM1_CAPT ; vector 11: Timer1 Capture Handler
    jmp TIM1_COMPA ; vector 12: Timer1 CompareA Handler
    jmp TIM1_COMPB ; vector 13: Timer1 CompareB Handler
    jmp TIM1_OVF ; vector 14: Timer1 Overflow Handler
    jmp TIM0_COMPA ; vector 15: Timer 0 CompareA handler
    jmp TIM0_COMPB ; vector 16: Timer 0 CompareB handler
    jmp TIM0_OVF ; vector 17: Timer0 Overflow Handler
    jmp SPI_STC ; vector 18: SPI Transfer Complete Handler
    jmp USART_RXC ; vector 19: USART RX Complete Handler
    jmp USART_UDRE ; vector 20: USART UDR Empty Handler
    jmp USART_TXC ; vector 21: USART TX Complete Handler
    jmp ADC_conv ; vector 22: ADC Conversion Complete Handler
    jmp EE_RDY ; vector 23: EEPROM Ready Handler
    jmp ANA_COMP ; vector 24: Analog Comparator Handler
    jmp TWSI ; vector 25: Two-wire Serial Interface Handler
    jmp SPM_RDY ; vector 26: Store Program Memory Ready Handler
;
; END OF INTERRUPT VECTORS
;
RESET:
; Inizia il programma principale
;
; Inizializzazione dello stack pointer con l'indirizzo dell'ultima cella di RAM
```

```
    ldi mp,HIGH(RAMEND)
    out SPH,mp
    ldi mp,LOW(RAMEND)
    out SPL,mp
;
; Inizializzazione in uscita del bit 5 della porta B, che verrà utilizzata per ➤
  misurare la tensione di alimentazione

    ldi mp,0b0010_0000
    out DDRB,mp

; Imposto inizialmente spento il LED tramite il bit 5, abilito il resistore di ➤
  pull-up su tutte le altre linee programmate in ingresso
    in mp,DDRB
    ldi mp1,0b1111_1111
    eor mp,mp1
    andi mp,0b1101_1111
    out PORTB,mp
;
; Inizializzazione in uscita dei primi 7 bit della porta D. PD0 - PD3 = ABCD; PD4 ➤
  = LE centinaia; PD5 = LE decine; PD6 = LE unità.
    ldi mp,0b0111_1111
    out DDRD,mp
;
    ldi mp,0b0111_0000      ; mette ad 1 tutti i LE dei 4511, (condizione di ➤
      memoria)
    out PORTD,mp
;
; Divide per 16 la frequenza del clock interno (16 MHz), così da diminuire i ➤
  consumi e avere una f = 1MHz
    ldi mp,0b1000_0000
    sts CLKPR,mp           ; abilita la programmazione del prescaler
    ldi mp,0b0000_0100
    sts CLKPR,mp           ; programma la divisione per 16 del clock interno
;
; Programma il timer counter 0, selezione prescaler passo 1024 (se 1 MHz allora ➤
  1024 us)
    ldi mp,0b0000_0101
    out TCCR0B,mp
; Seleziona un tempo tra interrupt pari a circa 20 ms (20.48 ms)
    ldi mp,236
    out TCNT0,mp
; abilita l'interrupt in caso di overflow di TCNT0
    ldi mp,0b0000_0001
    sts TIMSK0,mp
;
; Inizializza la variabile var_B che sarà decrementata dalla subroutine di ➤
  risposta ad interrupt
    ldi var_B,T_B
```

```

...ice_progetto_gruppo_22\Codice_progetto_gruppo_22\main.asm 9
; Inizializza la variabile var_LUX che sarà decrementata dalla subroutine di ↗
risposta ad interrupt
    ldi var_LUX,T_LUX
; inizializza la variabile flags
;
    ldi flags, 0b0000_0000
;
; programma l'ADC in modo da abilitarlo senza abilitare l'interrupt e seleziona ↗
un fattore di prescaling pari a 4.
;
    ldi mp,0b1000_0010
    sts ADCSRA,mp
;
; programma l'ADC perché lavori con riferimento la Vref esterna di 3.3 V ↗
proveniente dal generatore di tensione (riferimento interno spento), giustifica ↗
a sinistra il risultato
; e senta l'input su PC0 (ADC0) per la tensione di batteria
;
    ldi mp,0b0110_0000
    sts ADMUX,mp
;
; Abilita gli interrupt a livello di SREG
    sei
;
main_loop:
;
; verifica se è trascorso un tempo pari all'intervallo di campionamento della ↗
tensione di batteria(var_B = 0)
    cpi var_B,0
; salto condizionato a next01 se var_B è diverso da 0
    brne next01
; le istruzioni da qui a end_loop vengono eseguite solo se var_B è pari a 0
;
    ldi var_B,T_B          ; reinizializza var_B
;
;
    call batteria          ; richiamo della subroutine batteria che campiona la ↗
tensione e controlla non sia inferiore alla soglia di 4.2 V
    nop
;
next01:
; verifica se è trascorso un tempo pari all'intervallo di campionamento della ↗
tensione (var_LUX = 0)
    cpi var_LUX,0
; salto condizionato a end_loop se var_LUX è diverso da 0
    brne end_loop
; le istruzioni da qui a rjmp main_loop vengono eseguite solo se var_LUX è pari a ↗
0
    ldi var_LUX,T_LUX      ; reinizializza nuovamente var_LUX come prima cosa

```

```
;
; prepara il passaggio dell'indirizzo iniziale della tabella alla subroutine come
;   variabile locale nello stack
;
;   ldi ZH,high(2*tabella)      ; inizializza ZH con la parte alta dell'indirizzo
;   della tabella
;   ldi ZL,low(2*tabella)      ; inizializza ZL con la parte bassa
;   dell'indirizzo della tabella
;
;   push    ZH                ; mette ZH nello stack per passarlo alla subroutine
;   che lo utilizzerà come parametro
;
;   push    ZL                ; mette ZL nello stack per passarlo alla subroutine
;   che lo utilizzerà come parametro
;
; richiamo della subroutine luxmetro che campiona la tensione e visualizza il
; risultato sul display a sette segmenti (3 cifre) come intensità luminosa in
; klux
;   call luxmetro
;
end_loop:
;
;   nop
;
;   rjmp main_loop
;
batteria:
;
; imposto in admux la tensione di riferimento dell'adc pari a 5v, inoltre devo
; andare a selezionare la porta ADC 0, ma mantenere la giustificazione a sinistra
;   ldi mp,0b0110_0000
;   sts ADMUX,mp
; inizia adesso la conversione portando a 1 il bit 6 di ADCSRA (ADSC) senza
; modificare null'altro in ADCSRA
;
;   lds mp,ADCSRA
;   ldi mp1,0b0100_0000
;   or mp,mp1
;   sts ADCSRA,mp
;
;
; aspetta che la conversione sia pronta testando ADSC in ADCSRA: a conversione
; terminata ADSC torna a 0 - La conversione
; richiede 13 cicli di clock dell'ADC che, con le scelte fatte rispetto al
; prescaler dell'ADC corrispondono a
; 13 x 4 microsecondi (52 us). Con questo loop di attesa non utilizza l'interrupt
; dell'ADC
;
check_conv:
```



```
    lds mp,ADCSRA
    ldi mp1,0b0100_0000
    and mp,mp1
    brne check_conv
;
; legge il valore convertito su 8 bit (siccome è stata scelta la giustificazione ➤
; a sinistra gli 8 bit più significativi
; del risultato sono in ADCH
    lds mp,ADCH
;
; se il valore letto è maggiore o uguale alla soglia, salta a next02
    cpi mp,TSH_BAT
    brsh next02
; mette ad 1 il bit 0 di flags senza modificare gli altri, servirà per far ➤
; accendere il LED nel caso il ciclo precedente fosse spento
    ldi mp1,0b0000_0001
    or flags,mp1
;
next02:
; se il valore letto è minore della soglia, salta a next03
    cpi mp,TSH_BAT
    brlo next03
; mette a 0 il bit 0 di flags senza modificare gli altri, servirà per far ➤
; spegnere il LED nel caso il ciclo precedente fosse acceso
    ldi mp1,0b1111_1110
    and flags,mp1
;
next03:
;
; verifica il flags per eventualmente accendere il LED
    ldi mp1,0b0000_0001      ; prepara la maschera per il bit 0 di flags
    and mp1,flags           ; isola il bit 0 di flags in mp1
    cpi mp1,0b0000_0001
    brne next04
;
; scrive il valore 1 in PB5, il LED si accende
    ldi mp,0b0010_0000
    out PORTB,mp
;
next04:
;
; verifica il flags per eventualmente spegnere il LED
    ldi mp1,0b0000_0001      ; prepara la maschera per il bit 0 di flags
    and mp1,flags           ; isola il bit 0 di flags in mp1
    cpi mp1,0b0000_0000
    brne end_batteria
;
; scrive il valore 0 in PB5, il LED si spegne
    ldi mp,0b0000_0000
```

```
    out PORTB,mp
;
;
end_batteria:
;
    nop
;
ret                ; ritorna al programma chiamante
;
;
;
luxmetro:
;
    pop    mp                ; estrae temporaneamente dallo stack l'ultimo byte  ↗
    ; dell'indirizzo di rientro messo nello stack dalla call
    pop    mp1               ; estrarre temporaneamente dallo stack il primo byte  ↗
    ; dell'indirizzo di rientro messo nello stack dalla call
    pop    ZL                ; recupera il byte basso dell'indirizzo della  ↗
    ; tabella che è stato passato dal programma chiamante nello stack
    pop    ZH                ; recupera il byte alto dell'indirizzo della tabella ↗
    ; che è stato passato dal programma chiamante nello stack
    push   mp1               ; ripristina nello stack il primo byte  ↗
    ; dell'indirizzo di rientro messo nello stack dalla call
    push   mp                ; ripristina nello stack l'ultimo byte  ↗
    ; dell'indirizzo di rientro messo nello stack dalla call
;
; programma l'ADC perché lavori con riferimento la Vref esterna di 3.3 V  ↗
; proveniente dal generatore di tensione (riferimento interno spento), giustifica ↗
; a sinistra il risultato
; e senta l'input su PC1 (ADC1) per la tensione di batteria
;
    ldi    mp,0b0010_0001
    sts    ADMUX,mp
; inizia adesso la conversione portando a 1 il bit 6 di ADCSRA (ADSC) senza  ↗
; modificare null'altro in ADCSRA
;
    lds    mp,ADCSRA
    ldi    mp1,0b0100_0000
    or     mp,mp1
    sts    ADCSRA,mp
;
; aspetta che la conversione sia pronta testando ADSC in ADCSRA: a conversione  ↗
; terminata ADSC torna a 0
;
check_conv2:
    lds    mp,ADCSRA
    ldi    mp1,0b0100_0000
    and    mp,mp1
    brne   check_conv2
```

```

;
; legge il valore convertito su 8 bit (siccome è stata scelta la giustificazione
; a sinistra gli 8 bit più significativi del risultato sono in ADCH)
;
    lds mp,ADCH
    ldi mp1,4
    mul mp,mp1
;
    add ZL,R0          ; punta alla cella della tabella che corrisponde al
    valore letto dall'ADC
    adc ZH,R1
;
; prepara la visualizzazione sul display
;
    lpm mp,Z+          ; legge il primo valore della riga della tabella
    che contiene il valore completo intero su 8 bit ed incrementa Z
    lpm mp,Z+          ; legge il secondo valore della riga della tabella
    che contiene la cifra delle centinaia ed incrementa Z per la lettura
    successiva
    ori mp,0b0111_0000 ; lascia ad 1 PD4 - PD7, bit relativi ai LE, e non
    modifica PD0 - PD3, bit relativi alla cifra da rappresentare
    out PORTD,mp       ; scrive in uscita la cifra delle centinaia
    andi mp,0b0110_1111 ; prepara PD4 a livello basso (LE centinaia) senza
    modificare la cifra delle centinaia
    out PORTD,mp       ; abilita LE centinaia
    nop               ; istruzioni di attesa necessarie per garantire
    l'efficacia dello strobe
    nop
    nop
    nop
    nop
    ori mp,0b0111_0000 ; mette ad 1 PD4 - PD7 (alza LE)
    out PORTD,mp
;
;
    lpm mp,Z+          ; legge il terzo valore della riga della tabella
    che contiene la cifra delle decine ed incrementa Z per la lettura
    successiva
    ori mp,0b0111_0000 ; lascia a 1 PD4 - PD7, bit relativi ai LE, e non
    modifica PD0 - PD3, bit relativi alla cifra da rappresentare
    out PORTD,mp       ; scrive in uscita la cifra delle decine
    andi mp,0b0101_1111 ; prepara PD5 a livello basso (LE decine) senza
    modificare la cifra delle decine
    out PORTD,mp       ; abilita LE decine
    nop               ; istruzioni di attesa necessarie per garantire
    l'efficacia dello strobe
    nop
    nop
    nop

```

```

    nop
    ori mp,0b0111_0000      ; mette ad 1 PD4 - PD7 (alza LE)
    out PORTD,mp
;
;
    lpm mp,Z+               ; legge il quarto valore della riga della tabella ↗
    ; che contiene la cifra delle unità ed incrementa Zt
    ori mp,0b0111_0000      ; lascia a 1 PD4 - PD7, bit relativi ai LE, e non ↗
    ; modifica PD0 - PD3, bit relativi alla cifra da rappresentare
    out PORTD,mp            ; scrive in uscita la cifra delle unità
    andi mp,0b0011_1111     ; prepara PD6 a livello basso (LE unità) senza ↗
    ; modificare la cifra delle unità
    out PORTD,mp            ; abilita LE unità
    nop                     ; istruzioni di attesa necessarie per garantire ↗
    ; l'efficacia dello strobe
    nop
    nop
    nop
    nop
    ori mp,0b0111_0000      ; mette ad 1 PD4 - PD7 (alza LE)
    out PORTD,mp
;
; Scrittura display terminata
;
ret                               ; ritorna al programma chiamante
;
; INTERRUPT HANDLERS FOLLOW
;
;
EXT_INT0:
    reti                     ; vector 2:      IRQ0 Handler
;
EXT_INT1:
    reti                     ; vector 3:      IRQ1 Handler
;
PCINTR0:
    reti                     ; vector 4:      PCINT0 Handler
;
PCINTR1:
    reti                     ; vector 5:      PCINT1 Handler
;
PCINTR2:
    reti                     ; vector 6:      PCINT2 Handler
;
WDT:
    reti                     ; vector 7:      Watchdog timer handler
;
TIM2_COMPA:
    reti                     ; vector 8:      Timer2 compare A handler

```

```
;
TIM2_COMPB:
    reti                                ; vector 9:      Timer2 compare B handler
;
TIM2_OVF:
    reti                                ; vector 10:     Timer2 Overflow Handler
;
TIM1_CAPT:
    reti                                ; vector 11:     Timer1 Capture Handler
;
TIM1_COMPA:
    reti                                ; vector 12:     Timer1 CompareA Handler
;
TIM1_COMPB:
    reti                                ; vector 13:     Timer1 CompareB Handler
;
TIM1_OVF:
    reti                                ; vector 14:     Timer1 Overflow Handler
;
TIM0_COMPA:
    reti                                ; vector 15:     Timer 0 CompareA handler
;
TIM0_COMPB:
    reti                                ; vector 16:     Timer 0 CompareB handler
;
;
TIM0_OVF:
;
;   salva mp nello stack prima di utilizzarlo
    push mp
;   salva SREG nello stack
    in mp,SREG
    push mp
;   inizializza nuovamente la variabile di conteggio del timer counter (20.48 ms)
    ldi mp,236
    out TCNT0,mp
;   decrementa la variabile di conteggio della temporizzazione dell'ADC per la ↗
    dec var_B
    dec var_LUX
;   ripristina SREG
    pop mp
    out SREG,mp
;   ripristina mp
    pop mp
;
    reti                                ; vector 17:     Timer0 Overflow Handler
;
;
```

```
;
SPI_STC:
    reti                ; vector 18:      SPI Transfer Complete    ↗
    Handler
;
USART_RXC:
    reti                ; vector 19:      USART RX Complete Handler
;
USART_UDRE:
    reti                ; vector 20:      USART UDR Empty Handler
;
USART_TXC:
    reti                ; vector 21:      USART TX Complete Handler
;
ADC_conv:
    reti                ; vector 22:      ADC Conversion Complete  ↗
    Handler
;
EE_RDY:
    reti                ; vector 23:      EEPROM Ready Handler
;
ANA_COMP:
    reti                ; vector 24:      Analog Comparator Handler
;
TWI:
    reti                ; vector 25:      Two-wire Serial Interface ↗
    Handler
;
SPM_RDY:
    reti                ; vector 26:      Store Program Memory     ↗
    Ready Handler
```